

k-Based Control Testing

- Evolution of Our Approach
- Industry Validation
- Risk and Compliance Collaboration
- Impact on External Audits and Regulatory Compliance
- Additional Resources

Overview

Risk-based compliance is our strategic approach to using risk intelligence for informing compliance activities and testing priorities, rather than treating all compliance requirements equally. At GitLab, we sometimes also refer to this as risk-based control testing to differentiate the ongoing continuous monitoring of our SaaS platforms to maintain our certifications/attestations. This strategy recognizes that not all systems, vendors, or controls carry the same level of risk to the organization.

Risk-Based Control Testing

Risk-based control testing focuses on evaluating the effectiveness of internal controls based on the risk they mitigate. By concentrating on controls associated with higher-risk functions, we ensure critical controls are functioning properly without expending unnecessary resources on lower-risk functions.

Key Aspects

1. Control Identification: Determining which controls are in place to mitigate identified risks
1. Testing Prioritization: Allocating testing efforts to controls that address the most significant risks
1. Evidence Collection: Gathering sufficient evidence to assess control effectiveness, with more rigorous testing for higher-risk controls

Evolution of Our Approach

While achieving and maintaining certifications for our SaaS platforms remains a core priority of our team, an effective security compliance program must look beyond certifications and audit results. Some of our most sensitive data and critical systems fall outside traditional compliance boundaries. This reality demands that our security policies, standards, and best practices are implemented comprehensively across the enterprise.

With finite resources, we've adopted a risk-based framework to optimize our efforts. This approach enables us to:

- Maintain our certification requirements
- Ensure proper safeguards for our crown jewels as well as customer (RED) data
- Allocate resources based on actual risk exposure
- Adapt quickly to emerging threats and changes in our environment

This evolution from traditional compliance to risk-based compliance reflects our commitment to meaningful security outcomes rather than just "checking boxes" for auditors.

Industry Validation

Industry trends and best practices supporting this shift include:

- Moving away from checkbox compliance toward risk-informed testing
- Increasing use of automated continuous control monitoring
- Greater emphasis on data-driven prioritization
- Organizations increasingly adopting risk-based approaches to compliance management
- Development of shared risk/compliance taxonomies
- Regular joint risk-compliance working sessions

Risk and Compliance Collaboration

A beautiful synergy and feedback loop is created when Risk and Compliance collaborate together. Although we are completely separate teams with separate focuses, a risk-based compliance program allows us to:

- Establish formal coordination processes
- Share data and insights systematically
- Perform regular joint reviews of findings
- Provide unified reporting to leadership

Feedback Loop

mermaid

flowchart LR

```
subgraph Risk[Risk Team]
    R1[Risk Assessments]
    R2[Vendor Reviews]
    R3[Risk Register]
end
```

```
subgraph Compliance[Compliance Team]
    C1[Control Testing]
    C2[Control Effectiveness]
end
```

```
R1 --> |Updates| R3
R1 --> |Testing Priorities| C1
R2 --> |Testing Priorities| C1
R3 --> |Testing Priorities| C1
```

```
C1 --> |Testing Results| C2
C2 --> |Informs| R1
```

Risk Team's Input to Compliance

- Risk assessments identify highest operational risks (testing priorities)
- TPRM Assessments highlight critical third-party relationships

- Risk Register provides thematic view of operational risk across the organization

Compliance Team's Input to Risk

- Control testing results identify gaps and weaknesses
- Compliance findings map back to Risk Register items including risk treatment plans
- Testing coverage data informs risk response decisions
- Control effectiveness metrics are leveraged as Key Risk Indicators (KRIs) in Quarterly Risk Reporting

Benefits of Integration

- More efficient resource allocation focused on highest-risk functions
- Agility to quickly adapt to evolving threats and regulatory changes
- Stronger justification for compliance activities
- Better data to drive risk reporting accuracy and inform risk decision-making
- More comprehensive view of organizational security posture
- Reduced redundancy in assessment activities

Impact on External Audits and Regulatory Compliance

Our risk-based compliance program complements rather than conflicts with our formal audit and certification obligations. This approach is carefully designed to enhance our security posture while maintaining clear boundaries around regulated activities.

We maintain strict separation between:

- SOX compliance activities and associated Section 302 reporting
- Existing certification and attestation requirements (SOC 2, ISO 27001, etc.)
- Risk-based compliance activities focused on enterprise security

By explicitly excluding systems and controls in scope for SOX and other certifications/attestations from our testing scope, we ensure our risk-based testing does not interfere with or create confusion around formal audit activities. We maintain distinct documentation, testing schedules, and reporting processes for each domain.

Our findings and metrics remain internal to the security organization and inform our continuous improvement efforts. They are not:

- Reported to internal audit
- Shared with external auditors
- Used for regulatory compliance purposes
- Included in formal attestations

Note: While our program's findings are for internal security use, standard incident reporting obligations remain in effect should any testing identify an active security incident.

Additional Resources

The Security Compliance team maintains details on how we execute this program in our team issue

tracker. There you can learn how we:

- Prioritize which systems and applications to test
- Determine which controls are important
- Collaborate with Security Risk
- Share inputs and outputs between teams

SECTION: security/security-assurance/security-compliance/risk-based-control-testing.md

What is Risk-based control testing?

Risk-based control testing refers to control testing and monitoring that aims to ensure controls are operating effectively beyond the scope of GitLab's SOC 2 reports and information security management system for our SaaS platforms, which have foundational controls in place where customers need the most assurance.

Risk-based control testing is proactive and focuses on mitigating the most significant risks to the organization, providing a more effective and resource-efficient means of maintaining security. It is adaptive, driven by indicators such as GitLab's quarterly risk assessments, and aims to continually improve security posture.

In contrast, control monitoring of SaaS platforms is aligned with what external auditors test and is more standardized, compliance-driven, and typically occurs at fixed intervals to verify that controls meet the requirements of a specific standard (SOC 2, ISO 27001, PCI DSS, etc.).

Reporting on control and system health beyond external certification maintenance provides a clearer picture of overall control health across the organization.

Why test controls that are not included in external certifications?

As a security compliance team and key part of the 2nd line of defense, we are responsible for ensuring security controls are operating effectively across the entire company, and compliance posture communicated to leadership to ensure proper attention and allocation of resources can be considered. We need to provide risk mitigation value beyond just certification maintenance.

By focusing on high-risk controls, the compliance team can optimize its resources, ensuring the most critical risks are managed effectively without unnecessarily allocating resources to low-risk areas.

External certifications (for example SOC2, SOX, FedRamp, etc) target specific elements/sections of the environment relevant to the framework.

In the example of SOX, this means just systems that support financial/accounting processes. For SOC2, this could mean anywhere customer data is stored or parts of the environment directly involved in the product customers use. The environment at GitLab is more expansive than just these segments of the environment, meaning risk-based control testing allows us to measure and assess controls for other higher risk systems or processes that are not tested or monitored already, giving us a more in-depth view of control health.

How are the controls selected for testing?

Controls are prioritized based on various risk assessments and impact assessments that we already do as a company.

To select controls for risk-based control testing, we consider the following attributes:

- Critical Systems Tier per the tech stack
- Crown Jewels per Threat Intelligence in the Internal Handbook
- Data Classification per the tech stack
- Risk associated with SaaS/Vendors (for example, a recent breach)
- Coverage of identified risks (such as risks identified by the Security Risk Team, CIS Top 18 Critical Security Controls, known gaps)
- Baseline Level of control per NIST 800-53B
- Time since control was last tested
- Historical Observations
- Nature of the control (e.g. manual, semi-automated, automated)
- Regulatory guidance
- Changes in people, process or technology

Because some controls are already being tested through the Control Monitoring process, these controls will not be prioritized for testing. Testing strategies adapt over time as new risks emerge or controls are deemed to be functioning effectively. It's dynamic, taking into account risk changes based on internal changes (for example, system updates) or external factors (for example, new regulatory requirements).

The team will coordinate quarterly to identify the controls to be tested for the next quarter based on the above criteria.

How often are controls tested for risk-based control testing?

While the Security-Compliance will perform risk-based control testing on a quarterly basis, the frequency of a particular control's testing will also be based on risk. Ideally, testing will be balanced between the results of previous testing and expanding our insight into the control environment as a whole. If we determine a control is designed and operating effectively, and the process is subject to few errors, we may elect to test a different control/system rather than covering the same control each quarter.

How do we report on results of risk-based control testing?

(to be updated - <https://gitlab.com/gitlab-com/gl-security/security-assurance/security-compliance/team/-/issues/275>)

Risk-based Control Testing Requests

If you have a process or control that you think should be tested based on the criteria above that we might not be covering already, please follow these steps. Note: to have an entire system (new or changing) assessed, please see the Security System Intake process.

1. Confirm that the controls are not already tested

(to be updated - <https://gitlab.com/gitlab-com/gl-security/security-assurance/security-compliance/team/-/issues/267>)

2. Create a Security Compliance Intake Issue

(to be updated - <https://gitlab.com/gitlab-com/gl-security/security-assurance/security-compliance/team/-/issues/267>)

3. Security Compliance workflow

(to be updated - <https://gitlab.com/gitlab-com/gl-security/security-assurance/security-compliance/team/-/issues/267>)

SECTION: security/security-assurance/security-compliance/sbom-plan.md

Purpose

The purpose of this page is to provide direction on where GitLab needs to go as both a software producer and software consumer to provide transparency, and most importantly visibility, into the security of our software supply chain. GitLab is evaluating a very dynamic regulatory landscape with references to SBOM requirements in numerous draft legislation in both the U.S. and non-U.S., executive orders and directives, customer requests, and numerous community frameworks and formatting specifications.

Why SBOMs

The ability to generate and consume complete and accurate SBOMs is essential for securing our software and managing our software supply chain risk. Here are a few reasons why we should prioritize it.

1. Transparency is one of our core values

- Providing complete and accurate information the components used in our software, beyond name and version to include pedigree and vulnerability exploitability data, is directly in line with our values and the fact that we are an open core company. SBOMs are the vehicle through which software transparency can be achieved.

1. Lack of visibility into software dependencies and vulnerabilities is a top risk for every company

- The information in SBOMs from other companies and open source projects helps GitLab make quick, risk-informed decisions.

1. Competitive differentiation

- The world is still coming to terms with the reality of SBOMs, both present and future. As a DevSecOps category leader, GitLab will be held to a higher standard than our competitors and will need to be leading the world in this domain from both a product perspective and a customer trust perspective.

1. Efficiency

- Conveying information in a standard, authoritative format such as CycloneDX and VEX means that we and our customers will gain numerous efficiencies. For example - there will never be questions/requests for which dependencies are actually present in software and which

vulnerabilities are exploitable and their status.

1. Regulatory mandates

- Although there are few binding requirements today, SBOMs have been referenced as a requirement for U.S. Federal Agencies to do business with software providers in an Executive Order, the National Cybersecurity Strategy, NIST SSDF standard, CISA, and is already appearing in draft legislation and Requests for Information (RFI). Non-U.S. public sector and commercial regulatory frameworks are following suit, and the regulatory landscape is expected to very dynamic over the coming years. See the Resources section at the bottom of this page for more information.

Current State (as of 16.4)

Refer to GitLab's Software Supply Chain Security direction page for the latest information on current and planned features related to SBOMs.

GitLab the Product currently has the following SBOM features: ability to generate CycloneDX-formatted SBOMs after GitLab Dependency and Container scans, and the ability to view a Dependency List with vulnerability and license data pulled in from existing GitLab databases. As of 16.4, GitLab the product added an API for Pipeline-specific CycloneDX SBOM exports and the ability to create group-level dependency lists.

GitLab the Company uses the Dependency List for viewing vulnerability information for key projects, however generally uses the Vulnerability Report instead after running dependency and/or container scans in CI pipelines. Although CycloneDX SBOMs are generated for most of our projects, we do not yet consume these, combine them with one another, generate VEX BOMs, nor make them readily available to customers.

SBOM Maturity Model (work in progress)

Foundational Capabilities

GitLab the Product

- Ability to automatically generate complete and accurate SBOMs in an accepted format such as CycloneDX for GitLab repositories, which includes all of NTIA's recommended minimum data elements: Supplier Name, Component Name, Version of the Component, Other Unique Identifiers, Dependency Relationship, Author of SBOM Data, and Timestamp.

- Many of these fields are available today (sampled SBOM from a dependency scan job for gitlab-org/gitlab), however, Dependency Relationship and Author must be enabled. Feature request for missing fields.

- Feature for "Source SBOM" created directly from the development environment, source files, and included dependencies used to build a product artifact. Typically generated from software composition analysis (SCA) tooling, with manual clarifications.

- This is already a feature, see Dependency List docs. It cannot yet be exported into CycloneDX format.

- Feature for "Analyzed SBOM" generated through analysis of artifacts (e.g., executables, packages, containers, and virtual machine images) after its build. Such analysis generally requires a variety of heuristics. In some contexts, this may also be referred to as a "3rd party" SBOM. Typically generated through analysis of artifacts by 3rd party tooling.

- This is already a feature as CycloneDX SBOMs can be produced in CI pipelines for both

Dependency Scanning and Container Scanning.

- Support for digitally signing SBOMs at build or release time (release evidence) and/or upon being generated.
- This is already a planned feature reflected on our Software Supply Chain Security direction page under SBOM generation and management.

GitLab the Company

- Generate complete and accurate SBOMs for all product-related / released software for customers. Document instances where SBOMs are not complete/accurate (e.g. due to declaring dependencies in an unexpected way).
 - Export release SBOM for GitLab Omnibus
- Combine GitLab SBOMs to ensure they are representative of GitLab product releases, as opposed to one-off components/projects.
 - This can be done manually, for example using CycloneDX CLI, but there is no native GitLab support for this.
- Create a company-wide SBOM Standard which includes the practices and process that must be followed such as the tools and format, how frequently we will generate SBOMs and for which software/projects, what level of depth it needs to be, how it will be used/consumed, how we will track inaccuracies or incompleteness, and how they will be distributed/shared with customers.
- Digitally sign all SBOMs and/or assemblies within BOMs to provide integrity verification and non-repudiation.
- Maintain readily accessible and digitally signed SBOM repositories, and have a documented mechanism to share SBOMs with customers. The mechanism may be within the Security CAP or through a similar, publicly available page. Inform Product, Field Security, and Sales of how customers can request/access GitLab SBOMs.

Intermediate Capabilities

GitLab the Product

- Ability to generate and annotate a VEX BOM (or embedded VEX data within the SBOM) for a linked SBOM.
 - Support VEX in SBOM Reports
- Feature for "Build SBOM" generated as part of the process of building the software to create a releasable artifact (e.g., executable or package) from data such as source files, dependencies, built components, build process ephemeral data, and other SBOMs. Typically generated as part of a build process. May consist of integrated intermediate Build and Source SBOMs for a final release artifact SBOM.
 - This is already a planned feature reflected on our Software Supply Chain Security direction page under Release Evidence Generation.
- Include additional data elements on GitLab's SBOMs to better inform overall risk posture and expanded use cases. This includes provenance and pedigree information for dependencies, links to VEX BOMs, license information, hash for each component, and purl/cpe/swid references.
- Include metadata for generating a dependency graph such as bom-ref for every component.
- GitLab has a feature to import 3rd party SBOMs (ideally within GitLab using UI and API) and enrich this with vulnerability data (e.g. from GitLab Security Advisory database).
- Include security advisories as external references within SBOMs / VEX BOMs.
- Develop a feature to combine SBOMs within GitLab to get a full picture of a particular

component/assembly or product. For example this could be achieved by manually selecting various SBOM job artifacts and/or dependency lists from within a top-level group.

- Native support for other SBOM formats such as SPDX. Customers can already convert CycloneDX outputs into SPDX via a custom CI job or via publicly available OSS tools.

GitLab the Company

- GitLab has a mechanism (e.g. contract) to hold 3rd party commercial vendors accountable for the same standard that we set for ourselves regarding acceptable SBOM format, level of detail, frequency, completeness/accuracy, etc. This includes ensuring SBOM availability and accuracy as part of our Third Party Risk Management program.

- Maintain vendor vulnerability disclosure reports at the SBOM component level.

- GitLab has a process to consume and analyze SBOMs (ideally within GitLab using UI and API) from 3rd party commercial vendors and non-GitLab open source software to understand if there are vulnerability or licensing risks.

- GitLab maintains up-to-date vulnerability metadata for its SBOMs via annotating and updating linked VEX BOMs (or embedded VEX metadata). The intent is to relay VEX info to customers.

- Automatically consume and report on vulnerabilities gleaned from GitLab-generated SBOMs and linked VEX BOMs at a point in time. The intent is to drive internal security awareness and remediation prioritization.

- Produce and maintain a SaaS SBOM for GitLab.com and GitLab Dedicated.

- Produce and maintain a SBOM for GitLab Omnibus and CNG.

Advanced Capabilities

GitLab the Product

- GitLab has a feature to import 3rd party VEX BOMs to make it possible to view vulnerability metadata from 3rd party and other open source software directly in GitLab.

- Automatically keep VEX BOMs up to date by extracting remediation status information using pedigree metadata.

- Support for SBOM assemblies to show relationship of nested software components/products and transitive dependencies.

- Provide support for other properties in GitLab-generated SBOMs that allow for automating the triage and remediation process (e.g. using built-in name-value store to identify product groups or departments, internal severity information, remediation SLAs/due dates, etc.).

- Provide support for composition completeness information within the SBOM.

- ML Transparency metadata (once supported by industry-accepted specifications such as CycloneDX).

- Support Vulnerability Disclosure Report (VDR) as a feature in SBOM reports. Issue: Support Vulnerability Disclosure Report (VDR) in SBOM reports.

- Feature for "Deployed SBOM" provides an inventory of software that is present on a system. This may be an assembly of other SBOMs that combines analysis of configuration options, and examination of execution behavior in a (potentially simulated) deployment environment. Typically generated by recording the SBOMs and configuration information of artifacts that have been installed on systems.

- Feature for "Runtime SBOM" generated through instrumenting the system running the software, to capture only components present in the system, as well as external call-outs or dynamically loaded components. In some contexts, this may also be referred to as an "Instrumented" or "Dynamic" SBOM. Typically generated from tooling interacting with a system to record the

artifacts present in a running environment and/or that have been executed.

GitLab the Company

- Develop risk management and measurement capabilities to dynamically monitor the impact of SBOMs' vulnerability disclosures on the acquiring organization.
- Perform binary decomposition of software installation packages to generate SBOMs when no vendor-supplied SBOM is available (e.g., legacy software), when technically and legally feasible.
- Ensure all available SBOM spec properties are completed for GitLab's SBOMs including vulnerability metadata/VEX, composition completeness information, assembly data, and other properties as needed.
- Add Vulnerability Disclosure Report (VDR) data to CycloneDX SBOMs.

Proposed SBOM Implementation Plan / Roadmap (not yet agreed to)

FY 25

Target implementation of all Foundational capabilities and select Intermediate capabilities for both GitLab the Product and GitLab the Company.

FY 26

Target implementation of all Intermediate capabilities and select Advanced capabilities for both GitLab the Product and GitLab the Company, as warranted by customer value-add, product differentiation, or security/compliance requirements.

Resources

SECTION: security/security-assurance/security-compliance/scap-scanning.md

Security Content Automation Protocol (SCAP) Scanning

Configuration Compliance of IT systems is an important part of system hardening and is part of most security attestations and third party audits. Security Content Automation Protocol (SCAP) is a common method of automating these configuration compliance checks and identifying remediation.

Background

Security Content Automation Protocol is a US standard maintained by NIST, and is a method for using specific standards to enable automated vulnerability management, measurement, and policy compliance of information systems. It is most often used for compliance scanning, that is to measure IT systems against specific compliance standards like CIS or PCI-DSS.

OpenSCAP is a very popular open-source ecosystem for SCAP scanning and is maintained by Red Hat. It is the most common way to conduct SCAP scanning and assessment.

SCAP modules

Each SCAP (Security Content Automation Protocol) module has a specific function to help automate security checks, assessments, and vulnerability management. Here is a breakdown of the key SCAP modules and what each one does:

1. XCCDF (Extensible Configuration Checklist Description Format)

- Purpose: XCCDF is used to create security checklists, benchmarks, and configuration guidance.
- Function: It defines security policies as XML documents that include rules, checks, and remediation guidance. XCCDF files are structured to represent security profiles, with each profile containing rules for configuration and compliance.
- Example Use: A benchmark to verify that SSH settings align with best practices would be defined in an XCCDF file.

2. OVAL (Open Vulnerability and Assessment Language)

- Purpose: OVAL provides a standard way to represent system configuration information, state, and vulnerability data.
- Function: OVAL definitions describe security checks for specific configurations, patches, or vulnerabilities. It allows scanners to evaluate system compliance and find vulnerabilities by checking if specific conditions are met.
- Example Use: An OVAL check might confirm that a particular version of a package is installed, or that a configuration file contains certain values.

3. CPE (Common Platform Enumeration)

- Purpose: CPE is a standardized naming scheme to identify information technology systems, platforms, and applications.
- Function: CPE names uniquely identify specific platforms, operating systems, or applications, enabling SCAP tools to apply relevant checks based on the target system.
- Example Use: A CPE identifier might specify a system as "cpe:/o:redhat:enterpriselinux:8" to apply checks specific to Red Hat Enterprise Linux 8.

4. CVE (Common Vulnerabilities and Exposures)

- Purpose: CVE provides a unique identifier for publicly known vulnerabilities.
- Function: Each CVE ID refers to a specific vulnerability, describing the affected software and the nature of the issue. CVE identifiers help track vulnerabilities across databases and security tools.
- Example Use: CVE-2021-34527 (PrintNightmare) is an identifier that refers to a specific Windows vulnerability, allowing security tools to recognize and reference it in scans and reports.

How These Modules Work Together

- XCCDF defines what should be checked and provides remediation guidance.
- OVAL performs the actual checks by evaluating system state and configurations.
- CPE identifies the target systems to apply the correct XCCDF and OVAL rules.
- CVE provides a way to reference specific vulnerabilities in security content and scan

results.

These modules collectively enable SCAP tools to perform consistent and automated security assessments across diverse systems and platforms.

How this applies to GitLab systems

At GitLab we primarily deploy our production application through containers. OpenSCAP can be ran against a wide variety of deployment options, but mainly it is ran against operating systems and container images. We run OpenSCAP against our infrastructure operating systems and container images to validate their compliance.

OpenSCAP cannot evaluate container images on it's own and requires a container engine like Podman to do so.

Getting Started with OpenSCAP Compliance Scanning

1. Choose your module based on your intent (XCCDF, OVAL, CPE, CVE)
2. Source your SCAP content (for example, XCCDF files are part of the ssg family of packages)
3. Decide on your standard (PCI-DSS, CIS, CUI, etc)
4. Check your SCAP content for the relevant OS standard file.
 1. RedHat 9 Example: /usr/share/xml/scap/ssg/content/ssg-rhel9-ds-1.2.xml
5. For compliance scanning, get the profile name of the standard you want to use.
 1. You can find the list within the SCAP content file using `oscap info [filename]`
 1. e.g. `oscap info /usr/share/xml/scap/ssg/content/ssg-rhel9-ds-1.2.xml`
6. put together your command and run it against your system or container image.
7. IMPORTANT: `oscap-podman` can only run as root (`sudo oscap-podman` or `sudo su - then oscap-podman`), and podman is designed to isolate container images and configurations between users. You need to pull the containers as the root user or using `sudo` so that `oscap-podman` can see the container image.

Container image example: CIS L2 benchmark for RedHat 9 against a UBI9-minimal image using `oscap-podman`.

bash

```
COMMAND CONTAINERIMAGE MODULE MODE --report DESIREDREPORTLOCATION --profile STAND
LOCATIONOFSCAPCONTENTFILE
```

```
oscap-podman c8d01adc0698 xccdf eval --report /tmp/ubi9-min-scap-cis-2.html --profile
xccdforg.ssgproject.contentprofilecis /usr/share/xml/scap/ssg/content/ssg-rhel9-ds-1.2.xml
```

This evaluates the container image `c8d01adc0698` against the CIS benchmark that exists in `ssg-rhel9-ds-1.2.xml` and outputs the file to `/tmp/`

Practical Steps

We will:

1. Setup OpenSCAP on an Ubuntu 24.04 system

2. evaluate the RedHat 9 UBI image against the CIS benchmark

Prep the system

On an Ubuntu 24.04 system:

1. Install Podman (included in the default apt repositories in 24.04)
2. install openscap-scanner openscap-utils openscap-common openscap-doc
3. Install ssg-base ssg-nondebian
4. check /usr/share/xml/scap/ssg/content for SCAP profiles
 1. These are installed by ssg-base and ssg-nondebian
5. There is a bootstrap script here for ease of setup.

SCAP Evaluation Commands

bash

Become root so the command works (alternatively use sudo for each command)

sudo su -

Pull the latest container image

podman pull registry.access.redhat.com/ubi9/ubi

check for the container image ID

podman image ls

grab the container image ID and run your command

oscap-podman a674bd5e68d9 xccdf eval --report /tmp/ubi9-min-scap-cis-2.html --profile

xccdforg.ssgproject.contentprofilecis /usr/share/xml/scap/ssg/content/ssg-rhel9-ds-1.2.xml

Open the final HTML file in your browser

Other Points

- OpenSCAP can be used for a variety of standards and formats. For example, US Department of Defense Security Technical Implementation Guides (STIGs) are some of the strictest configuration standards that are publicly available. STIGs can be used with OpenSCAP.
- OpenSCAP can generate different kinds of reports and scripts, not just HTML. Depending on the specific system it can generate the exact commands necessary to remediate failures.
- If JSON output is needed, this OSCAP-JSON tool will intake the standard output of openscap and convert it into JSON. If using an ARM architecture you will need to build it from source using make.
- This is a working example script that uses the oscap-json tool to generate JSON SCAP reports from a list of containers.

References

- this video from RedHat
- Red Hat article on OpenSCAP
- OpenSCAP website: <https://www.open-scap.org/>
- The source of the ssg family of packages and other SCAP content: <https://github.com/ComplianceAsCode/content>

SECTION: security/security-assurance/security-compliance/sec-controls.md

Security controls are a way to state our company's position on a variety of security topics. It's not enough to simply say "We encrypt data" since our customers and teams will naturally want to know "what data do we encrypt?" and "how do we encrypt that data?". When all of our established security controls are operating effectively this creates a security program greater than the sum of its parts. It demonstrates to our stakeholders that GitLab has a mature and comprehensive security program that will provide assurance that data within GitLab is reasonably protected.

GitLab Control Framework (GCF)

We have tried to take a comprehensive approach to our immediate and future security compliance needs. Older and larger companies tend to treat each security compliance requirement individually which results in independent security compliance teams going out to internal teams with multiple overlapping requests. For example, at such a company you might have one database engineer that is asked to provide evidence of how a particular database is encrypted based on SOC2 requirements, then again for ISO requirements, then again for PCI requirements. This approach can be visualized as follows:

```
mermaid
graph TD;
  SOC2Requirement1-->Team1;
  SOC2Requirement1-->Team2;
  SOC2Requirement2-->Team1;
  SOC2Requirement2-->Team2;
  PCIRequirement1-->Team1;
  ISORequirement1-->Team2;
```

Given our efficiency value here at GitLab we wanted to create a set of security controls that would address multiple underlying requirements with a single security control which would allow us to make fewer requests of our internal teams and efficiently collect all evidence we would need for a variety of audits at once. This approach can be visualized as follows:

```
mermaid
graph TD;
  SOC2Requirement1-->Control1;
  PCIRequirement1-->Control1;
  SOC2Requirement2-->Control2;
  ISORequirement1-->Control2;
  Control1-->GCF;
  Control2-->GCF;
  GCF-->Team1;
  GCF-->Team2;
```

As our security compliance goals and requirements have evolved so have our requirements and constraints related to our security control framework. Our GCF is currently based on NIST SP

800-53 and is in the process of being iterated into a custom proprietary framework that aligns to GitLab's needs and security posture. This transformation is necessary due to the fact that not all NIST 800-53 controls map directly to the requirements and standards we maintain.

The custom framework will maintain mappings to all our compliance requirements (SOC2, ISO, PCI, etc.) while eliminating irrelevant controls and adding GitLab-specific controls where needed. This new framework is scheduled to be implemented starting in Q3 FY26, providing us with a more streamlined and relevant security control structure.

Security Control Lifecycle

The lifecycle of our security controls can be found at this handbook page. As part of the security control lifecycle, GCF security controls are monitored or tested based on risk or as required by regulation.

GCF security controls are assessed at the entity, shared process, or system level depending upon the nature of the control.

Risk-based Control Testing is our methodology for determining the appropriate frequency and depth of control testing based on the criticality and risk profile of a system. This approach allows us to allocate testing resources more efficiently and expand testing beyond what's in-scope for external audits by focusing more on higher risk systems while maintaining appropriate coverage across all systems. Controls associated with systems are evaluated based upon the system's critical system tiering.

Control Ownership

Control Owner - Ensures that the design of the control and the control activities operate effectively and is responsible for remediation of any control activities that are required to bring that control into a state of audit-readiness.

Process Owner - Supports the operation of the control and carries out the process designed by the control owner. The process owner is most likely to be interviewed by an auditor to determine whether or not the process is operating as intended.

Security System Intake

To assess newly acquired/developed systems that enable security controls OR are/may be in scope for compliance programs for potential inclusion into our GitLab Control Framework (GCF) and compliance programs (e.g., Security Compliance Program and SOX Program).

1. System identification

Our goal is to identify systems that enable security controls (e.g., access management system) OR systems that are (or may be) subject to regulatory (e.g., SOX) or compliance requirements (SOC2, PCI) as early as possible. As we develop new features/systems and engage with third parties for services, we assess the use of the system and whether or not it meets the criteria described above. Existing systems can also be ingested into the Security Compliance Intake process. Examples of these could include systems whose functionality has expanded to support security controls or instances where our understanding of a security control has improved resulting in the identification of a previously uncredited supporting system.

If the system meets the criteria, we open up a new Security Compliance Intake Issue.

2. Creating Security Compliance Intake Issue

Security Compliance Intake Issue asks the author to include details related to the system including:

- System overview
 - Implemented security controls or impacted regulatory or compliance program
 - Link to TPRM reviews or Production Readiness reviews
 - Critical System Tier (CST)
 - Key Contact Details
 - Deployment model and implementation status
 - Data classification
 - Regulatory scope (supported by Security Compliance and Internal Audit)
- The author of the issue completes as much of the issue as they can and assigns it to the Security Risk team for completion/triage (if the issue is not originally created by the Security Risk team).

3. Security Compliance workflow

Once the Security Compliance Intake issue is populated, Security Risk assigns the issue to the Security Compliance team to complete the following tasks to incorporate the system into our Security Compliance Program:

- Determine impact (if any) to compliance control environment (for example, determining if applicable to the PCI environment and need for inspecting PCI AoC and Responsibility Matrix documentation)
- Notify stakeholders and system owners of upcoming testing requirements
- Incorporate testing requirements (driven by CST and regulatory/compliance requirements) and recommendations into the fiscal year audit schedule
- Determine when user access reviews for the new system need to start and communicate to compliance teams

References

[Return to the Security Compliance](../../security-compliance/)

SECTION: security/security-assurance/security-compliance/security-control-lifecycle.md

Process Overview

{: width="600px"}

Purpose

As new GitLab security controls are identified that need to be implemented by the Security

Compliance Teams for compliance or regulatory reasons, these controls follow an established process in order to make that implementation successful.

These lifecycle phases are managed via GitLab's governance, risk and compliance (GRC) application.

Scope

This document applies to GitLab's security controls being assessed by the Security Compliance Team(s).

Roles & Responsibilities

| Role | Responsibility|

| ---- | ----- |

| GitLab Team Members | Responsible for following the requirements of the controls |

| Security Compliance Team | Responsible for execution of security control tests and maintenance of this handbook page |

| Security Compliance Management | Responsible for oversight, escalation and approval of exceptions for this process |

| Security Assurance Management (Code Owners) | Responsible for approving significant changes and exceptions to this procedure |

Procedure

Lifecycle Phases Explained

Preparation

As new GCF security controls are identified they first must be contextualized to GitLab as a company and to the applicable GitLab systems. The Preparation phase of the control lifecycle covers this initial work required to get controls into a state of ready to be tested.

Additionally, GCF controls that have been previously tested but have an upcoming requirement for renewed testing enter this Preparation phase as well to research and confirm that any changes to the control processes are captured in the updated testing activity.

Testing

mermaid

```
graph TD;
  1[Test of Design]-->2[Decision: TOD Pass];
  1-->3[Decision: TOD Fail];
  3-->9[Observation];
  2-->4[Test of OE];
  4-->5[Decision: TOE Pass];
  5-->6[Operating];
  4-->7[Decision: TOE Fail];
  7-->9[Observation];
  9-->8[Remediation];
```

The testing activity consists of 3 major components:

1. Assessing the Design and Operating Effectiveness of the control

1. Validating observations (if any were noted during testing) with the observation owners

1. Recording those observations (if any) according to the Security Compliance Observation Management process

- Note: These observations can only be recorded after being validated by the observation owner to ensure that observation is accurate and represents a material deficiency in the security control process

After assessment and validation, the status field in the GRC tool should be updated as follows:

- If a gap exists with no current control implementation, create a placeholder control and list the status as 'gap'

- Controls with a status of 'gap' should have a corresponding observation issue.

- If a control was designed and operating effectively with no observations, create a control ID, control description, map it to the appropriate control domain and list the status as 'in existence'.

Remediation

Remediation is the phase of the lifecycle where required changes are made to the design of the security control or the process of the control's operation. Remediation is either performed by the observation owner or is tracked by the observation owner if the observation remediation is blocked by another GitLab team's work. The Security Compliance Team is responsible for tracking all validated observations and continually reporting on those observations to ensure they are tracked, prioritized appropriately, and escalated as needed to meet security compliance program goals.

Operating

Controls that are tested with no observations noted during that testing activity are determined to be in an operational state. This indicates that the design and operating effectiveness of this control are at or above the level required to meet the current needs of the security compliance program.

Controls in an operating state will still need to be re-tested as determined by the risk rating of the control to ensure no substantive changes have occurred which would impact the design or operating effectiveness of that control.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Controlled Document Procedure

[Return to the Security Compliance](../../security-compliance/)

SECTION: security/security-assurance/security-risk/_index.md

Mission Statement

To drive security risk treatment at GitLab by empowering teams to make informed and intelligent decisions through proactive identification, monitoring, prioritization, and reporting of security risks.

Value Proposition

We perform thorough, collaborative, and efficient risk assessments as well as drive risk reduction so that GitLab can achieve its goals while maintaining a high level of security.

Core Competencies

Security Operational Risk Management (StORM) Program{storm}

The Security Risk team manages an integrated Operational Risk Management program focused on the identification, assessment, continuous monitoring, and reporting of Security Risks across the organization. Risk Reduction is 1 of 5 of the Security Department's operating principles (Security Vision and Mission). As such, the Security Risk Team takes a leading role in providing the information required by leadership to establish our Strategic Roadmap and our support GitLab's key initiatives (internal only). Visit the StORM Program & Procedures handbook page for additional details including templates and how we integrate with other risk programs at GitLab.

{{% panel header="Need to communicate a potential risk to the team?" header-bg="warning" %}}
Please refer to the communication section of the StORM Program & Procedures page for information on the various ways that team members can use to escalate potential risks to the Security Risk Team.
{{% /panel %}}

Security Third Party Risk Management (TPRM) Program{tprm}

GitLab maintains an industry-leading Third Party Risk Management (TPRM) Program through the use of automation, continuous monitoring, and deep integration across business functions to validate the security of GitLab data shared with external parties. The integration of GitLab's TPRM program within the vendor Procurement flow enables cross-functional collaboration between Privacy, Legal, IT, and People Operations to facilitate transparent, risk-based decision making, Business and Stakeholder-focused Results, and adherence to GitLab's Regulatory and Compliance Obligations. The vendor relationships maintained through this program are leveraged to create efficiencies across the organization.

Business Impact Analysis (BIA) and Critical System Tiering (CST){bia}

The Business Impact Analysis (BIA) helps determine the systems critical to serving GitLab's Customers. The output of the BIA is the designation of a Critical System Tier (CST) for a new system by the Security Risk Team.

Asset Inventory Maintenance{asset-inventory}

Establishing a complete and accurate inventory of assets is key to the success of GitLab's Risk Program. As such, the Security Risk Team collaborates closely with IT and Business Owners to ensure new systems are added to the Tech Stack.

Operating Model

Core Processes

Function	DRI
:-----: :-----:	
Annual Risk Assessment	Kyle Smith
Business Impact Analysis	Nirmal Devarajan
New System Additions to Tech Stack and Post-Implementation Checks	Nirmal Devarajan
Critical System Tiering	Kyle Smith
Ongoing SecRisk-Related Observations Management	Nirmal Devarajan
Ongoing Risk Treatment	Kyle Smith
Ongoing TPRM Assessments	Ryan Lawson
Periodic SOX CUEC Facilitation	Eric Geving
Periodic TPRM Assessments	Eric Geving
TPRM Data Quality and Emerging Requirements Management	Eric Geving
StORM Metrics and Reporting	Kyle Smith
TPRM Metrics and Reporting	Ryan Lawson
TPRM Application Integrations	Ryan Lawson

Engagement Models

1. Request a TPRM Review
1. Report Security Operational Risk
1. In GitLab, tag the team across GitLab using @gitlab-com/gl-security/security-assurance/security-risk-team

Communication Channels{contact}

1. *<i class="fas fa-envelope fa-fw" style="color:rgb(219,59,33)" aria-hidden="true"></i>* Email: securityrisk@gitlab.com
1. *<i class="fab fa-slack fa-fw" style="color:rgb(219,59,33)" aria-hidden="true"></i>* Slack: sec-assurance channel (includes the broader Security Assurance Team) - Mention @security-risk
1. *<i class="fab fa-gitlab fa-fw" style="color:rgb(219,59,33)" aria-hidden="true"></i>* GitLab: Tag the team across GitLab using @gitlab-com/gl-security/security-assurance/security-risk-team - Mention @security-risk

Team Members{team-members}

Team Member	Role
-------------	------

|:-----:|:-----:|
 |Ty Dilbeck|Senior Manager, Security Risk|
 |Nirmal Devarajan|Senior Security Risk Engineer|
 |Eric Geving|Senior Security Risk Engineer|
 |Ryan Lawson|Senior Security Risk Engineer|
 |Kyle Smith|Staff Security Risk Engineer|

Success Metrics

| Key Metric | Why It Matters | How it's Calculated | Target Thresholds | Measurement Frequency
Reporting Mechanism	Additional Notes					
Top 5 Risk Reduction over Time	The Top 5 risks represent the top 5 security risks to GitLab the company. These risks need to be reduced and managed effectively to reduce the likelihood of a significant security incident	Risk scores are calculated based on impact and likelihood.	Risks are considered in tolerance with a risk score of 10 or below.	Quarterly	Tableau Dashboard (internal only)	n/a
Third Party Risk Management Capacity	An indicator of third party risk, third party risk assessments proactively identify potential vendor security risks as part of onboarding or contracting, enabling business owners to make risk based decisions throughout the vendor lifecycle.	Count of total number security assessments completed QoQ	Less than 70 per quarter.	Quarterly	TPRM Metrics Sheet (internal only)	Will be added back into Tableau in FY26Q2

FY26 Strategic Initiatives

| | Objective | Key Deliverables | Timeline |
 |:~:| :----- | :----- | :-----: |
 | 1 | Enhance our Acceptable Use Policy Program | - Block unnecessary or insecure integrations

 - Define processes for new integrations
 - Restrict privileges to install integrations without security and privacy reviews | Ongoing, FY26-27 |
2	Consolidate Security Division Issues/Recommendations	- Inventory of all sources of recommendations from the Security Division - Centralized view of all issues/recs. across Security Division - Standardization across issues/recs. with documentation standards - Scalable management of issues/recs. to improve adoption	In progress Target Comp.: Q3
3	Corporate and Product Disaster Recovery Planning and Governance	- Collaborate with Business Teams to plan and govern the execution of BC/DR control activities to fulfill external certification requirements - Perform RTO/RPO analysis for critical vendor systems, escalate gaps/discrepancies compared to RTO/RPO recommendations, and draft a resolution plan	In progress; Target completion: Q2-end
4	Update Security Assurance KPIs	Develop a comprehensive Security Assurance metrics dashboard to monitor, measure, and validate that all security process success criteria are being effectively tracked and achieved	In progress; Target completion: Q2

Review and Updates

This charter will be reviewed and updated quarterly to ensure alignment with:

1. GitLab Strategy
1. Security Division Mission and Vision

1. Security's Multi-year Strategy (internal only)
1. Security Assurance Mission and Vision
1. Security Assurance Multi-year Strategy (internal only)

Next scheduled review: 2025-09-15

SECTION: security/security-assurance/security-risk/sox_cuec_mapping_procedure.md

Purpose

In accordance with ITGC SR.1 - SOC Report Review, GitLab executes annual CUEC mappings of our internal controls to each SOC report associated with a SOX in scope application to ensure controls are adequately designed to address the CUEC requirements outlined in the SOC report. This activity is executed in Q1 of each fiscal year in order to gain the greatest coverage for the prior fiscal year.

Scope

Formal CUEC mappings are limited to SOX in scope third party SaaS applications as defined by management.

Coverage

All third party SOX system's SOC1 reports and bridge letters providing coverage over the prior fiscal year (2/1/20xx-1/31/20xx) will be obtained. If SOC1 reports are unavailable, SOC2 reports will be obtained.

Roles & Responsibilities

Role	Responsibility
Security Risk - TPRM	Executes collateral collection and vendor interface - Creates CUEC Mapping templates and populates with CUECs & CSOCs
Internal Audit	Defines scope of SOX systems and facilitates mapping activities
Business Owners	Participates in mapping activities and provides final approval

Procedure

Collateral Collection (Security Risk)

As Security Risk is responsible for executing third party security risk management activities the team will support CUEC activities through the gathering of necessary collateral, specifically SOC reports and bridge letters. Security Risk follows this process:

1. Validates the systematic scope by leveraging IAs SOX listing
1. Creates a GitLab Epic for activity tracking and transparency/visibility. This epic can be modeled after prior year.
1. Initiates requests and obtains the applicable third party reports and bridge letters

1. Reviews SOC reports for CSOCs and initiates requests to obtain all CSOC reports. Note that Security Risk does not determine whether or not GitLab business processes rely on CSOCs. As such, we request all CSOC SOC reports.

1. Validates that reports provide coverage over the prior fiscal year (2/1/20xx-1/31/20xx). If coverage is lacking additional requests are sent.

Throughout this process, Security Risk manages the status of requested reports and CUEC mapping template creation prior to handoff to IA.

Google Sheet CUEC Template Creation (Security Risk)

To help formally hand-off the CUEC mapping activity, Security Risk creates CUEC mapping templates and adds CUECs/CSOCs from in-scope SOC reports. Security Risk follows this process:

1. Creates CUEC mapping templates in Google Sheets for each in-scope report (including CSOCs)
1. Lists all CUECs within the template
1. Shares the CUEC mapping templates and applicable reports with Internal Audit

ITGC and Business Process Control Mappings (Business Owners/Internal Audit)

Once updated SOC report are obtained, Internal Audit will:

- Work with the Business Owner to map internal controls to the CUECs
- Coordinate with business owners to review SOC report findings and evaluate impact on business and ITGC controls
- Coordinate with business owners to review user control considerations to ensure appropriate GitLab controls are in place to address user control considerations
- Manage the status of business owner approvals

Approvals (Business Owners)

At conclusion of the mapping activities the Business Owners must provide final approval of the mapping. This activity is managed by Internal Audit.

Exceptions

N/A

References

- GitLab SOX ITGC Compliance
- Third Party Risk Management

SECTION: security/security-assurance/security-risk/third-party-risk-management.md

GitLab's Integrated Third-Party Risk Management Program

GitLab maintains an industry-leading Third Party Risk Management (TPRM) Program through the use

of automation, continuous monitoring, and deep integration across business functions to validate the security of GitLab data shared with external parties.

The integration of GitLab's TPRM program within the vendor Procurement flow enables cross-functional collaboration between Privacy, Legal, IT, and People Operations to facilitate transparent, risk-based decision-making, Business and Stakeholder-focused Results, and adherence to GitLab's Regulatory and Compliance Obligations. The vendor relationships maintained through this program are leveraged to create efficiencies across the organization.

Scope

This procedure applies to all third party providers that access, store, process or transmit GitLab data.

Purpose

GitLab's Security Third Party Risk Management (TPRM) Program helps guard against security threats posed by third parties who have access to GitLab data or that of our customers. These risks may include data breaches, unauthorized use or disclosure, and corruption or loss of data. Adequate TPRM is a best practice that helps mitigate security concerns and enables GitLab to meet our contractual obligations. TPRM also enables GitLab to meet regulatory requirements and standards related to ISO, SOX, GDPR and other state and federal laws requiring vendor oversight.

GitLab's Security TPRM program involves three components which are integrated into our Procurement processes:

1. Performing due diligence to ensure the third party implements safeguards to enforce data privacy and security
 - This activity is performed via our Security Assessment Process. You can find an open source version of our TPRM Assessment Report template in our Open Source Security Hub.
1. Contractually obligating the third party to implement these safeguards
1. Monitoring third party safeguards and compliance with these contracted provisions
 - Certain high risk Third Parties are reviewed annually while those posing lower risk are reviewed at intervals defined further within this document.

Roles & Responsibilities

Role	Responsibility
Security Risk Team	Maintain a mechanism to intake and respond to TPRM Activities Assess Third Party inherent and residual security risk Inform Business Owners of the result of TPRM assessments
Business or System Owner	Describe the nature of the Third Party Relationship Work with the Security Risk team to facilitate the TPRM review, to include remediation activities Ensure the responsiveness of the third party as part of the security review requirements
Security Assurance Management (Code Owners)	Responsible for approving significant changes and exceptions to this procedure

On-Call Rotation

TPRM engineers are assigned on-call duties on a weekly basis to manage vendor intake and TPRM inquiries. Team members are encouraged to tag the on-call engineer via Slack or within GitLab issues for urgent inquiries. The on-call calendar can be found [here](#).

After-Hours Support

The Security Risk team's core working hours are 8:00AM to 4:00PM CST. For urgent requisitions or TPRM concerns requiring immediate attention outside of these hours, please reach out to Ty Dilbeck via the phone number listed in Slack.

Frequently Asked Questions

| Question | Answer |

| ----- | ----- |

| What is the purpose of TPRM? | Our procedures exist to support informed business decisions and mitigate exposure to the ever-evolving threat landscape. This is critical to maintain customer trust and prevent unauthorized data exposure.|

| What is my role? | Your primary role as a requestor is to help us understand what is being purchased and what data is shared. This helps us complete an accurate assessment efficiently to support your needs.|

| You reviewed them last year, why are we doing it again? | Our assessments are valid for 12 months. After that, a new assessment is required before we can approve additional requisitions. A vendor's security environment and our compliance obligations are subject to change, thus continued oversight is necessary to stay aligned with evolving risks.|

| Can you just approve and assess after? | No. Introducing new vendors and systems prior to review can expose us to unforeseen security risks and compliance issues. If a request must proceed to address a critical business or customer need, approval may be granted via our Security Notice process.|

| Can this be expedited? | Our team can prioritize reviews within reason if a requisition is critical and time-sensitive, however the best way to expedite a review is to encourage the vendor to respond to our inquiries quickly. Urgent requests should be escalated to @Security-Risk in the Procurement channel.|

| How can I track review status? | Updates will be provided via comments within the Zip requisition. Status updates may be requested there or within the Procurement channel.|

| What if they fail the review? | A Security Notice will be opened to communicate deficiencies to relevant stakeholders and determine next steps to support the business need. New vendors that fail our TPRM review may be required to address any identified deficiencies before proceeding. For existing vendors, a remediation plan or offboarding plan may be required depending on the severity of the deficiency.|

|What triggers a TPRM Assessment? | Funny you should ask...|

What Triggers a TPRM Assessment?

Security Risk monitors various inputs to maintain compliance and ensure proper oversight. TPRM assessments are typically valid for 12 months unless otherwise specified, and are triggered in the following scenarios:

- New Vendor Engagement: New vendors or systems accessing Orange or Red data require a TPRM review prior to approval.
- Vendor Renewal: Vendor renewals may require a TPRM assessment prior to approval if the most

recent assessment has expired.

- Changes in Services: New services and functionality from existing vendors may introduce risks not covered by previous assessments.
- Security Incidents: An assessment may be triggered, along with a Technical Security Validation, if a vendor suffers a security incident potentially impacting GitLab data.
- System Integrations: Integrations between new or existing systems, especially those processing higher-sensitivity data, may require an assessment. See System Integrations for details.
- Annual High-Risk Reviews: Vendors in-scope for compliance testing (SOC2, SOX, etc.), GitLab Sub-Processors, and vendors otherwise processing Red data are reviewed annually to meet compliance obligations.

Security Considerations for Vendor Selection

Security is a top priority in GitLab's vendor selection process, and GitLab is committed to partnering with vendors who prioritize the protection of sensitive data and maintain robust security measures. As such, Security Risk thoroughly evaluates each vendor's security practices, including their adherence to industry standards, compliance with relevant regulations, and health of their Bitsight security ratings. GitLab's aim is to establish partnerships with vendors who share our commitment to maintaining the highest standards of security to safeguard GitLab's data and uphold the trust of our stakeholders. Below are some examples of how Security Risk evaluates vendors of different types and some recommendations for vetting vendors when making purchasing decisions.

<details>

<summary markdown="span">Software as a Service (SaaS)</summary>

When considering new software to address business needs and objectives, see below for some high level recommendations to leverage when evaluating the software's capabilities.

- Does the vendor support Single Sign-On (SSO) to enable Team Members to authenticate using their network credentials?
 - Security requires that software has the ability to integrate with Okta or Google Oauth for authentication.
- Does the vendor complete an annual independent third party audit of their security controls to show adherence to industry standards and best practices?
 - Security requires software vendors complete and share an annual independent third party audit of their security controls. Some examples of these are a SOC 2 or ISO 27001 certification.
- Does the vendor incorporate Secure by Design principles throughout the software development lifecycle?
 - Security requires that software vendors perform predeployment threat modeling, vulnerability scanning, and secure coding practices during design and development phases of the software.
- Does the vendor require an independent third party penetration test of their systems to identify and remediate vulnerabilities and weaknesses?
 - Security requires that vendors conduct annual independent penetration testing on all systems supporting their SaaS solution and attest to the remediation or mitigation of any medium or higher severity findings.

Note: Proof of concept (Poc) and pilot software engagements may require the exchange of sensitive GitLab data and must adhere to the same evaluation of the vendor's security practices for new software requests. Security Risk recommends anonymizing sensitive GitLab data and ensuring contract language defines expectations and requirements around data security, privacy considerations, and deletion of data after the PoC or pilot is complete.

</details>

<details>

<summary markdown="span">Professional Services</summary>

GitLab has an obligation to ensure that all individuals accessing Orange or Red data are subject to sufficient endpoint controls, either by issuing a GitLab-managed device or by performing a TPRM review to ensure appropriate endpoint security controls are in place. Professional services firms are often able to provide sufficient documentation of their security environment, as such we can rely on their endpoints once sufficient assurance is obtained. Independent contractors and firms without sufficiently documented endpoint controls will be provided GitLab-managed endpoints by IT.

See below for high level recommendations to leverage when evaluating professional services vendors.

- Will GitLab documents be shared with the vendor?
 - Security recommends using GitLab's Google Drive to share sensitive data with external parties.
- Does the professional services vendor adhere to strict human resource security practices including acceptable use and data protection training, background screenings, and regular security awareness trainings?
 - GitLab requires professional service vendor